HATCHABOT

Your AI should be your own staff, not a search box.

An **open-source agentic operating system**: many purpose-built agents · on hardware you own · thinking with the AI you choose · reachable by the people you let in · through Telegram · on one AI account.

OpenClaw + **Docker** + **Telegram** = **Hatchabot**

the open-source agent runtime · one isolated container per agent · the front door everyone already has

THE FIRST QUESTION

“Can't ChatGPT or Claude already do this?”

They remember you now. They run tasks on a schedule. They read your Drive. For one person in one app they are excellent — and Hatchabot uses one of them as its brain. Four things change once the assistant has to serve a household.

Everyone gets agents

Sharing an assistant there means everyone signs in — their own ChatGPT account, or a paid Claude seat. Here the family messages a Telegram contact: no app, no AI account, no second subscription.

The memory is yours

Both remember you, and let you view, edit and export it — in their app. Here it's a plain file on your disk the agent reads directly: back it up, run it under another model, move it.

A staff, not a generalist

One assistant answers your taxes, the homework and the condo board in one voice from one context. These are separate by design — own persona, model, data, people — and consult each other mid-task.

Your machine, scoped

Their tasks run in their cloud, with their tools, across your whole account. Each agent here gets exactly what its job needs: this folder, that repo, mail it may read but never send.

If you're the only one using it, the chat app is the better deal — polished, and no setup. Hatchabot is for several jobs, several people, and things you'd rather keep at home.

THE IDEA

The opposite shape

Many agents, each with a job

A kitchen helper, a tax advisor, a homework tutor, a scheduler that reads its own inbox. Made in a minute: a name, a paragraph, a bot token.

On a computer you own

Every agent is a container on your machine. Memory, credentials and history stay home. Pair it with a local model and your conversations never go to an AI company.

With the AI you choose

Claude (an API key, or your own plan), OpenAI, Gemini or a local model — per agent; models switch instantly. You buy **inference** from a lab and nothing else: how the agent behaves is open-source code and files on your disk.

For the people you let in

Invite family by link or QR. Talk to one agent one to one or together in a group room, with one shared memory — and everyone told so.

Through Telegram

The app everyone already has. Each agent is a real contact with a name and a face. Nothing to install, no AI account to create. Slack and Discord work too.

And it runs itself

Hatchabot has its own agent: a manager you talk to in plain words. It reads the fleet and hands back a card you confirm — and keeps each agent on the least capable model that does its job.

OpenClaw + Docker = Hatchabot. Every agent is a stock OpenClaw gateway — the open-source agent runtime — in its own Docker container with its own volume, bot and credentials. Hatchabot is OpenClaw *new and improved for a household*: many isolated installs, run as a fleet, by more than one person.

COMPARED

Side by side

A chat app is a place you go to talk to one assistant. Hatchabot is a staff of agents that live on your machine and come to you.

	CLAUDE.AI / CHATGPT	HATCHABOT
Shape	One assistant, one app, one context	**A staff of agents with jobs, for a household**
Who can use it	You; others need their own account, and a paid seat on Claude	**Anyone you invite, on Telegram, with no account of their own**
Memory	Kept by the provider; you can view, edit and export it, but it lives in their app	**Text files on your disk: read them, fix them, back them up, take them with you**
Acts on its own	Scheduled tasks: the assistant does the task each run, mostly in their cloud	**Schedules, events, requests from other agents — on your machine; a job can be a plain program, with the model only where needed**
What it costs	A plan per person, or per seat	**One AI source for every agent; each agent's cost on its tile, and a budget if you want one**
Access to your things	The connectors they built, across your whole account	**Per agent: one folder, one repo, one account — with limits you set**
Choice of AI	That company's models	**Claude, OpenAI, Gemini or a local model, per agent; models switch instantly, a new provider takes a rebuild**
Who runs it	They do — you get the app they ship	**An agent of yours does: ask it, confirm its cards, and it never holds the authority**
If you leave	Your history stays with them	**Export it, move machines, change provider — the agents are yours**

The chat apps are polished, need no setup, and their models are excellent — Hatchabot doesn't compete with the model, it uses one. Coding agents (Codex, Claude Code) are a different tool again: one agent, one repo, one task — and one of the things you can build here. Describes the consumer apps as of October 2026.

WHAT YOU BUY FROM A FRONTIER LAB

Inference. That's the whole list.

A chat app is a product built around a model, and whoever owns the model owns the product: how the agent behaves, what it remembers, which tools it may touch, what it costs next year — and whether it still exists next year.

They sell tokens

That is the entire dependency. An API key (or your own plan), called by the unmodified vendor binary or API.

Swapping labs is a setting, not a migration: an agent moves between Claude, Gemini and a local model without being rewritten.

The community owns the rest

OpenClaw is the open-source runtime; Hatchabot is the open-source control plane. MIT, both.

How an agent thinks, what it remembers, what it may reach and who may talk to it are files on your disk and code anyone can read, fork and fix.

Which is the point

OpenAI shut down its Assistants API in Aug 2026 and retires Agent Builder in Nov. Products built on a vendor's harness went with them.

Agents built here outlive any lab's roadmap — and any lab's pricing decision.

Rent the intelligence. Own the machinery.

TWO DELIBERATE CHOICES

One AI account and a front door people already have

1 account

One AI account, the whole household

Connect an Anthropic API key (billed per use) or a local model (nothing per message). The credential stays on your machine and reaches each agent through a private file — never a shared login — and each agent's tile shows what it costs.

The kids' tutor, the family planner, your own advisors: **everyone gets their own agents on one account**. Nobody else needs an AI account — they message the agents made for them.

Have your own Claude Pro or Max plan? You can connect it on your own machine instead; that use is between you and Anthropic, under their terms, which can change.

20 bots

Telegram — no app to build

Free, on every platform, and bots are a first-class feature: BotFather makes one in a minute, and a free account carries about 20 identities. Slack and Discord work too, one small app per agent. Every chat app connects outward — nothing of yours is exposed. An agent can also have no messaging app at all and answer in the Hatchabot app.

One AI account and a free front door: a household of agents, with what each one costs in plain sight.

BUILD ON IT

An operating system for agents

An agent is a container with a job, credentials and a memory of its own, made by describing it in English. The first takes an afternoon; the twentieth takes a minute. Some of the shapes that makes possible:

An agent anyone can email

A meeting scheduler with its own inbox: it negotiates a time with every invitee and runs group votes. Nobody needs Telegram or an account.

One agent per client

A property manager per condo corporation, with that client's mail and files — cloned for the next. Nothing mixes.

Agents that check agents

A QA agent wired to the one it tests, running regressions after every change.

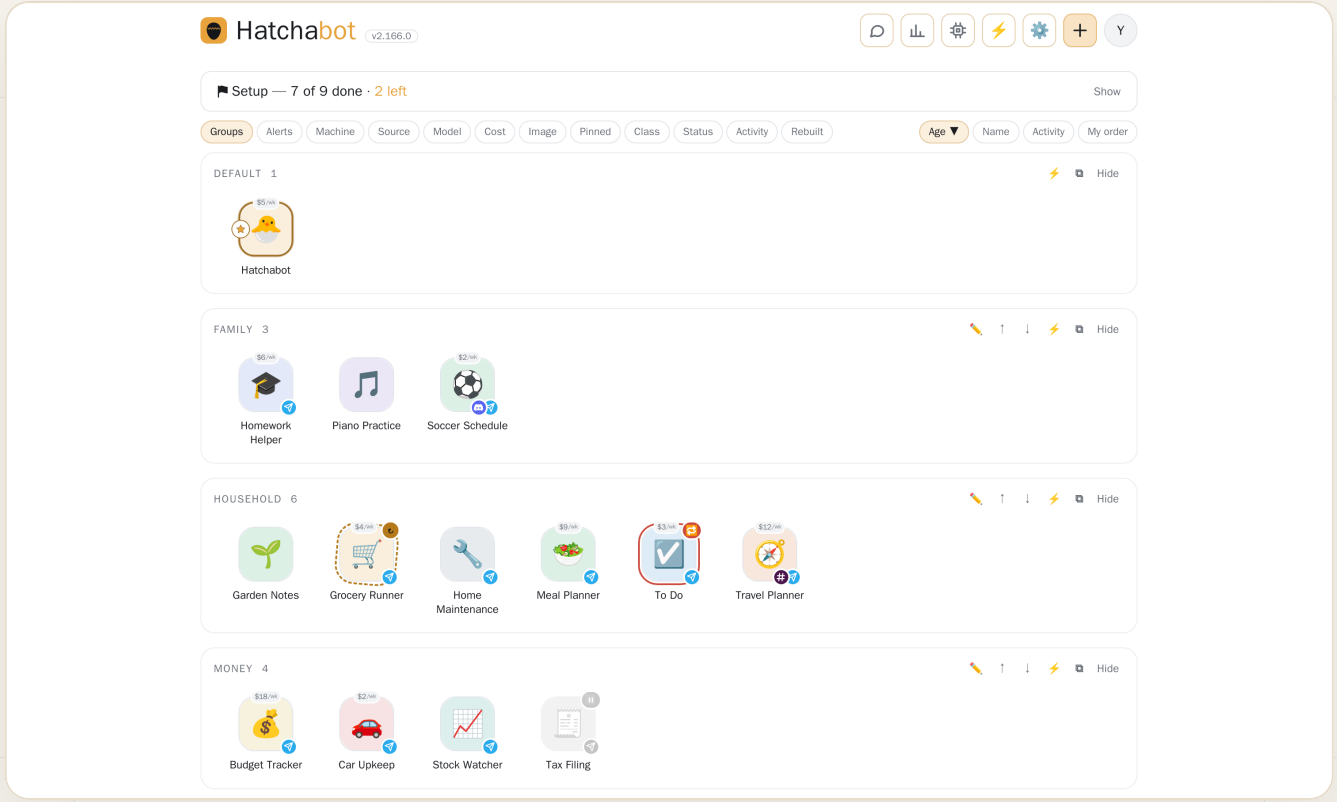
An agent you hand over

Export it as one file, credentials stripped; someone else runs it on their own accounts.

Honestly: OpenAI and Anthropic sell excellent engines. Neither hands you the layer around many long-lived agents — tenancy, credentials on your own hardware, a human front door, one console for the fleet.

THE CONTROL PANEL

Your whole staff on one page



An example fleet — the names and activity are invented for this picture.

Tiles in groups

Every agent is a tile you name, colour and drag into groups. The tile carries its state — rebuilding, stopped, unread, needs you — and a mark per app that reaches it.

The manager is the first tile

Your Hatchabot agent sits first, fixed in place, with a gold star. Ask it to make, fix, move or share an agent; it reads everything and changes nothing — each change is a card you confirm under Alerts.

View by, and what each costs

Regroup the page by Alerts, Model, Activity, Cost and more. A quiet chip shows a tile's last week at API prices — “$4/wk” — and Cost lists the fleet most expensive first. Usage charts what the agents spent, on what and on which model.

THE HATCHABOT SUPERVISOR

A supervisor for every agent's AI use

Your Hatchabot agent — the first tile, with the gold star — keeps each agent on the **least capable model that does its job well**, watches how agents spend tokens, and proposes the fix. You confirm every change.

Right model, on evidence

A weekly scorecard per agent: purpose, tool use, failures, cost on each model its source offers. Thin evidence or a small saving: no change.

A quality guard

Every model change is recorded. If failed turns or tool errors rise on the new model, Hatchabot proposes switching back.

Cost and savings, shown

Every tile shows its week's cost — “$4/wk” — and a Cost view ranks the fleet. “Saved by cheaper models: ≈ $12.40 this month” on Usage and atop the weekly report.

Loops, caught as they happen

A message retried again and again, a task failing run after run, agents consulting in circles: flagged in Alerts and on Telegram. A failing task: a card to pause it.

Conversations kept in size

Every message re-sends the whole conversation. When one grows large, it proposes a compaction now, or a cap so the agent compacts on its own.

A weekly token review

Cache, scheduled tasks, instruction files, thinking. **Budgets:** told at 80% and 100%; a cheaper model or a pause at the limit.

Ask it in its chat: “Which agents cost the most, and why?” · “Is anything on Opus that doesn't need to be?” · “Is anything stuck in a loop?”

ChatGPT's Auto picks a model per message, inside OpenAI's app, by its own rules; Claude's apps have you choose. Here it's per agent, under your control, with the savings shown.

Dollar figures are estimates: measured tokens at Anthropic's list API prices (on a Claude plan, room in its limits rather than money). The figures in quotes are examples.

USE THE MODEL AS LITTLE AS POSSIBLE

From about $1,700 a month to cents

A household's meeting-and-voting agent schedules meetings and runs board votes by email. It was an always-on AI agent. It is now a small program that runs inside a Hatchabot agent.

≈ $2,700

Before, a month at API prices

Every inbox check, every reply read and every decision was a model turn: about $1,700 for the agent and $990 for the QA agent that ran 49 regression cases against it.

Python

After: decisions in code

Parsing, scheduling, counting, sending and records are code. Claude Haiku is asked only where judgement on free-form email is needed, through the agent's own AI source. A quiet minute runs no model at all.

117 tests

Tested like software

Unit tests including the old QA agent's 49 cases, a check of its prompts against the real model, and a scripted live test against the real mailbox and calendar. The same email gets the same answer.

Cents a month. The agent gives the program a home: a container, the owner's connections, an AI source, Telegram for notices, a chat that runs its commands.

Before-figures are measured tokens at Anthropic's list API prices; the agent ran on a Claude plan, so they were room in the plan's limits rather than a bill. Since v2.144.0 such a program deploys into an agent from its repository, with its own tests run inside the agent first, and updates or rolls back with one command.

NEW SINCE 7 OCTOBER · V2.145 → V2.166

What it learned to do lately

Recommended, one list

Ranked cost, model and token advice on Usage: the concern, its evidence, the expected saving a month, one button. The manager's proposals sit in it, marked as its own.

Changes that survive a restart

Moves, imports and restores run on the server and carry on through a restart: finished or undone where the direction is certain, otherwise your choice under Alerts.

Recoverable, and shown

Each agent says when it is recoverable from and when its last drill passed, with a guided Restore. Drills run in isolated throwaway volumes, weekly or daily.

What each agent can reach

Accounts, bots, folders, repos, variables, people — what Hatchabot gave it beside what was found inside the running agent. Problems first, and under Alerts.

Apps and browsers, in the app

Run an app from a repo from New agent; Update runs its tests first, Roll back goes back. An agent can have a Chromium of its own, off by default.

Smaller things you will notice

Context now on Usage; Files that sort and open in the tab; a Test button on every folder and repo; Planned agents; Archive at the foot of Overview; OpenClaw 2026.9.8.

Each one is in the change log, with the release and the date it shipped.

FEATURES

What a staff can do that an assistant can't

CAPABILITY	IN PRACTICE
A new agent in a minute	A name and one paragraph → a running, remembering agent. Clone one that works, or import a template someone shared.
Many humans, one agent	Several people on the same agent, one to one or in a shared room. Invite by link or QR. One shared memory, declared to everyone.
A different brain per job	Claude, OpenAI, Gemini or a local model; switch live; classes retune a whole tier at once; migrate every agent off a source in one step.
Your accounts and files, scoped	Per agent: Gmail, Calendar, Drive and Sheets (with “read but never send”), read-only or writable folders, git repos it commits to — public ones need no key at all.
Work while you sleep	Scheduled tasks, and event-triggered ones where a zero-cost check decides whether to wake the model at all.
Agents that consult each other	Your investing agent asks your tax and legal agents mid-task, with scoped tokens, loop guards and rate limits.
Reachable where people are	Telegram, Slack or Discord — or no chat app at all. Each agent has its own bot, and strangers who find it get silence.
A manager and supervisor	Ask in plain words; every change is a card you confirm. It keeps each agent on the right model with the savings shown, and catches loops and oversized conversations.

Each agent gets only what its job needs — which is what makes it safe to give them real work.

FEATURES · OPERATING A FLEET

Operator-grade, because agents that matter need it

See what's going on

- Fleet health: probed, not assumed
- Usage and cost per agent, at API prices
- An audit timeline of every change
- A daily security posture with diffs
- A legend showing every pending action

Change things safely

- Bulk actions by source, group, class or state
- Classes: model + source + runtime image, retuned in one place
- Runtime images: build a candidate, try it on **one** agent, promote or discard
- Snapshots before every edit; rollback on failed moves

Keep it yours

- Nightly backups: every volume, the registry, the key
- Restore one agent from any backup set
- Export and import as a single file
- Runners: place agents on a second machine
- All over a private Tailscale network — nothing opened to the internet

Everything an operator needs — and nothing a family member ever has to see.

FEATURES · WHO GETS IN

Private by default, easy to share

The door rests shut

- A stranger who finds a bot gets **silence**
- Only people you invited — or who already use one of your agents — get through
- An invite can name the one person it is for
- Someone you already trust is added in one tap, with no pairing

Everyone signs in as themselves

- An account per person, each with their own agents
- Forgotten password: a one-time link you send them — you never see the new one
- “Forgot password?” reaches you through your linked Telegram
- No email server to set up

Set up by looking, not reading

- A setup guide that checks the machine itself
- Finds Tailscale and turns on HTTPS in one press
- Confirms the address actually answers
- A QR code to open it on your phone

A household AI has to be private and shareable at once — so the defaults do the first, and the app does the second.

FEATURES · OWNERSHIP

It's yours, all of it — and it fits in a file

You control every part of an agent

- Its **definition** — persona, rules, memory, in files you can read and edit
- The **tools** it has, and the **schedules** it runs on
- The **machine** it runs on — move it between yours
- The **AI** behind it — an API key, a local model, or your own Claude plan; models switch instantly
- The **data** it can reach — a folder, a repo, a mail account, scoped per agent
- **Who** can talk to it, and **which agents** it can consult

Download it all as one file

- A **complete copy**: definition, memory, members, credentials, even the recipe for its runtime image — restore it anywhere Hatchabot runs
- A **shareable template**: its instructions and schedules, with no bot, members or secrets (memory only if you choose), for someone to run on their own accounts
- Move an agent to another machine or another Hatchabot with one action; nothing is locked to a vendor or a login

Nothing about an agent lives anywhere you can't see, copy, edit or take with you.

PRINCIPLES

What we believe an agent is

A thing you own

A container with a persona, operating rules and a memory — plain files you can read and edit:

`SOUL.md` · `AGENTS.md` · `MEMORY.md`

It survives restarts, rebuilds, model switches and moves between machines.

Memory is a file, not a chat log

The agent writes durable facts as it goes. Save a conversation to memory, download its whole history, or recover the context of a thread that got reset.

Nothing important is trapped in a transcript.

Isolated by construction

One container, one bot, one set of credentials per agent — given exactly the data its job needs and nothing else.

A daily posture check names every agent that has both a wide audience and a powerful capability.

Reversible, or honest about it

Snapshots before edits and rebuilds. Rollback on a failed move. A full audit timeline. Health probes that tell “listed as running” from “actually answering”.

An agent is a folder of files you own, not a session you visit — and one agent failing never takes the others with it.

COMPARED

OpenClaw, new and improved: what Hatchabot adds

Every Hatchabot agent **is** an OpenClaw gateway. Hatchabot adds the layer OpenClaw deliberately leaves out: many installs, run as a fleet, by more than one person.

One install per agent, so agents can't hurt each other

Plain OpenClaw is one process, one config, one workspace and credential set shared by every agent in it. Hatchabot gives each agent its own container, volume, bot token, tool policy and secrets.

Upgrades the same way

The runtime image is pinned and rolled out per agent, candidate first — never “upgrade the gateway and hope every agent survives”.

Lifecycle instead of hand-editing

Create, clone, rebuild, archive, restore, delete, move — with a snapshot before every change. In plain OpenClaw that's you, a shell and a directory.

More than one human

Owners, members, invites, pairing approvals, group rooms, and one shared memory everyone is told about.

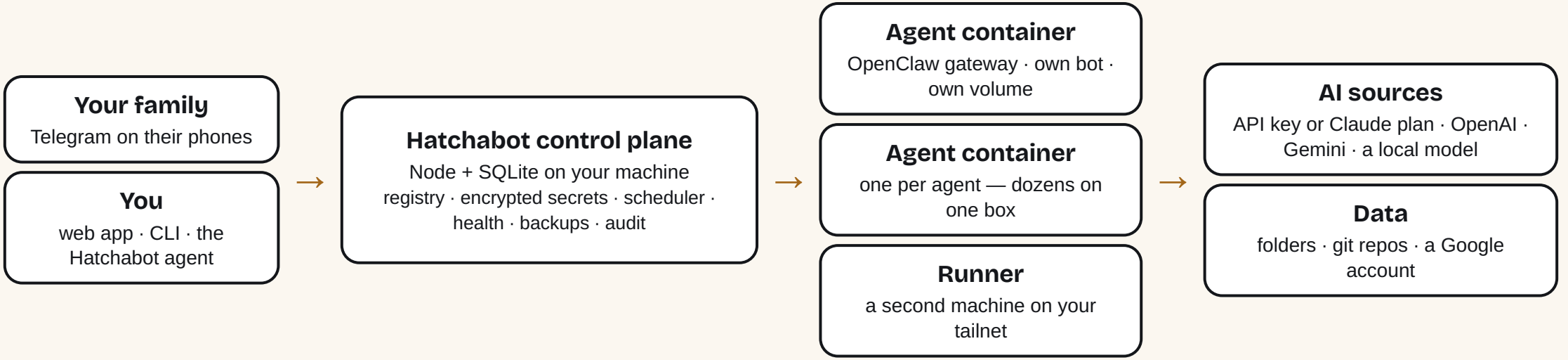
Credentials managed once

One AI credential serves every agent; Google accounts attach per agent; secrets are write-only.

If you run one agent for yourself, plain OpenClaw is fine. Hatchabot is for when there are several, they matter, and other people talk to them.

HOW IT WORKS

One control plane, one container per agent



Persisted

Each agent's `$HOME` is a Docker volume: persona, memory, sessions, tools. The runtime image is shared, versioned, and swapped under an agent without touching the volume.

Private by default

The web app lives on your machine; reach it from anywhere over Tailscale, never a port opened on your router. Telegram works from anywhere regardless.

Open source

TypeScript, MIT licensed, 3,500+ automated tests and 30+ full audits, the upgrade path tested in CI. Tagged releases with notes; pre-built images for arm64 and amd64.

GET STARTED

Your first agent in 15 minutes

1. **Make a Telegram bot (optional).** In Telegram, open @BotFather, send `/newbot`, and keep the token — or skip it and talk to your agents in the app.

2. **Get an AI key ready.** Create an API key at console.anthropic.com (billed per use). Or, if you have your own Claude Pro or Max plan, connect that instead: install Claude Code, sign in, and run `claude setup-token`. That use is between you and Anthropic, under their terms, which can change.

3. **Install Hatchabot** on the computer that stays on:

```
$ bash -c "$(curl -fsSL https://hatchabot.com/install.sh)"
```

4. **Open the app** at `http://localhost:8080` and create your account. The setup guide takes your API key (or token) and the bot token, makes your first agent, and can turn on HTTPS for your phone.

5. **Say hi** — click its icon, or message it on Telegram. It will still remember this conversation next year.

You need

- A Linux or macOS computer that stays on — a Mac mini, a home server
- Telegram — optional; agents also answer in the web app
- An AI: an Anthropic API key, a local model, or your own Claude Pro or Max plan

Made to be easy to run

- Only Docker needed on Ubuntu, Debian and Apple-silicon Macs
- The agent image is pre-built — a pull, not a build
- Ends with a link and a QR code; `hatchabot doctor` prints fixes

HATCHABOT

For people who want more than one assistant — shared with a household, on their own machine, with the AI they choose.

```
$ bash -c "$(curl -fsSL https://hatchabot.com/install.sh)"
```

hatchabot.com · github.com/hatchabot/hatchabot · MIT licensed